



JASHORE UNIVERSITY OF SCIENCE AND TECHNOLOGY

Department of Computer Science and Engineering

LAB REPORT

Course Code: CSE 3101

Course Title: Operating System and System Programming Lab

Lab Report No: 06

Lab Title: Implementation of Priority-Based CPU Scheduling Algorithm

Submitted To:

Monishanker Halder,
Assistant Professor,
Dept. of CSE, JUST

Submitted By:

Name: Adnan Zaman Niloy
Student ID: 210142
Session: 2021-2022
Year/Semester: 3rd Year, 1st Semester

Date of Submission: 21th July, 2025

Experiment Title: Implementation of Priority-Based CPU Scheduling Algorithm

1. Objective

The primary objective of this laboratory experiment is to implement the Priority Scheduling algorithm, a non-preemptive CPU scheduling technique that allocates CPU time based on process priorities. The goal is to understand how process priorities affect execution order, calculate waiting times, and analyze the algorithm's performance characteristics including the starvation problem.

2. Theory

2.1 CPU Scheduling Overview

CPU scheduling is a fundamental operating system function that determines which process should be executed next. The scheduler's primary goals include:

- Maximizing CPU utilization and throughput
- Minimizing waiting time and response time
- Ensuring fairness among processes
- Meeting system requirements and constraints

2.2 Priority Scheduling Algorithm

Priority Scheduling is a scheduling algorithm where each process is assigned a priority value, and the CPU is allocated to the process with the highest priority. Key characteristics include:

- **Priority Assignment:** Each process has an associated priority value
- **Priority Order:** Lower numerical values typically indicate higher priority
- **Non-preemptive:** Once a process starts executing, it runs to completion
- **Deterministic:** Execution order is completely determined by priorities

3. Algorithm Description

3.1 Priority Scheduling Steps

1. **Process Input:** Read all processes with their burst times and priorities
2. **Priority Sorting:** Arrange processes in ascending order of priority values
3. **Sequential Execution:** Execute processes in priority order
4. **Time Calculation:** Calculate waiting times and performance metrics
5. **Results Display:** Present scheduling results and statistics

3.2 Mathematical Formulations

- **Waiting Time (WT):** $WT[i] = \text{Sum of burst times of all higher priority processes}$
- **Turnaround Time (TAT):** $TAT[i] = WT[i] + \text{Burst_Time}[i]$
- **Average Waiting Time:** $AWG = (\text{Sum of all waiting times}) / n$
- **Average Turnaround Time:** $ATAT = (\text{Sum of all turnaround times}) / n$

4. Implementation

4.1 Source code

```
#include <bits/stdc++.h>
using namespace std;

struct Process
{
    string name;
    int burst_time;
    int priority;
    int waiting_time;
};

bool comparePriority(const Process &a, const Process &b)
{
    return a.priority < b.priority;
}

int main()
{
    int n;
    cin >> n;
    vector<Process> processes(n);

    for (int i = 0; i < n; i++)
    {
        cin >> processes[i].name >> processes[i].burst_time >> processes[i].priority;
    }

    sort(processes.begin(), processes.end(), comparePriority);

    int current_time = 0;
    int total_waitingtime = 0;
    for (int i = 0; i < n; i++)
    {
        processes[i].waiting_time = current_time;
        total_waitingtime += processes[i].waiting_time;
        current_time += processes[i].burst_time;
    }
}
```

```

    cout << "Process\tWaiting Time\n";
    for (const auto &p : processes)
    {
        cout << p.name << "\t" << p.waiting_time << endl;
    }

    cout << "average waiting time: " << fixed << setprecision(2) <<
(float)total_waitingtime / n << endl;

    return 0;
}

```

5. Test Case Analysis

5.1 Input Parameters

- **Number of Processes:** 5
- **Process Details:**

Process	Burst Time	Priority
P1	10	3
P2	1	1
P3	2	3
P4	1	4
P5	5	2

5.2 Priority-Based Sorting

Original Order: P1, P2, P3, P4, P5

Sorted Order (by Priority): P2(1), P5(2), P1(3), P3(3), P4(4)

Note: When processes have the same priority (P1 and P3 both have priority 3), the algorithm maintains their relative order from the input (stable sort behavior).

5.3 Step-by-Step Execution

Process Execution Timeline:

Step 1: P2 Execution (Time 0-1)

- **Process:** P2 (Priority: 1, Burst: 1)
- **Waiting Time:** 0 (first to execute)
- **Execution Period:** 0 → 1
- **Completion Time:** 1

Step 2: P5 Execution (Time 1-6)

- **Process:** P5 (Priority: 2, Burst: 5)
- **Waiting Time:** 1 (waited for P2)
- **Execution Period:** 1 $\rightarrow$ 6
- **Completion Time:** 6

Step 3: P1 Execution (Time 6-16)

- **Process:** P1 (Priority: 3, Burst: 10)
- **Waiting Time:** 6 (waited for P2 + P5)
- **Execution Period:** 6 $\rightarrow$ 16
- **Completion Time:** 16

Step 4: P3 Execution (Time 16-18)

- **Process:** P3 (Priority: 3, Burst: 2)
- **Waiting Time:** 16 (waited for P2 + P5 + P1)
- **Execution Period:** 16 $\rightarrow$ 18
- **Completion Time:** 18

Step 5: P4 Execution (Time 18-19)

- **Process:** P4 (Priority: 4, Burst: 1)
- **Waiting Time:** 18 (waited for all other processes)
- **Execution Period:** 18 $\rightarrow$ 19
- **Completion Time:** 19

6. Results Analysis

6.1 Process Execution Results

Process	Priority	Waiting Time
P2	1	0
P5	2	1
P1	3	6
P3	3	16
P4	4	18

6.3 Gantt Chart

0	1	6	16	18
P2	P5	P1	P3	P4

7. Algorithm Complexity Analysis

7.1 Time Complexity

- **Sorting Phase:** $O(n \log n)$ using efficient comparison-based sorting
- **Waiting Time Calculation:** $O(n)$ for sequential processing
- **Overall Time Complexity:** $O(n \log n)$ dominated by sorting operation

7.2 Space Complexity

- **Process Storage:** $O(n)$ for storing process information
- **Additional Space:** $O(1)$ for temporary variables
- **Overall Space Complexity:** $O(n)$ where n is the number of processes

8. Priority Impact Analysis

- **High Priority Advantage:** P2 has zero waiting time
- **Medium Priority:** P1 and P3 show significant waiting time difference due to processing order
- **Low Priority Penalty:** P4 experiences maximum waiting time (starvation risk)

15. Conclusion

The Priority Scheduling algorithm implementation successfully demonstrates the fundamental concepts of priority-based CPU scheduling. The experiment provides valuable insights into the trade-offs between process importance and fairness. However, it also highlights the risk of starvation for lower-priority processes if aging or other mitigation techniques are not applied.